



Operating Systems Lab

Course Code: CSE-412

Experiment Title: Shortest Job First (SJF) Non-Preemptive CPU Scheduling

Student Name: Md. Shoyaif Rahman

ID: 0432310005101050

Instructor Name: Afroja Ahmed Smrity

University Name: University of Information Technology and Sciences (UITS)

Experiment No: 03

Experiment Title: Implementation of SJF (Non-Preemptive) Scheduling

Date of Performance: 03/03/26

Date of Submission: 10/03/26

Objective of the Experiment

- To understand the SJF (Non-Preemptive) scheduling algorithm.
- To implement the algorithm in C++ considering Arrival Times (AT) and Burst Times (BT).
- To calculate CT, TAT, WT, and draw a Gantt chart.
- To compare SJF performance with FCFS.

Theory / Background

Shortest Job First (SJF) is a scheduling algorithm that selects the process with the smallest burst time for execution next. In its non-preemptive form, once a process starts execution, it continues until it finishes. SJF is optimal in terms of minimizing average waiting time, but it can cause starvation for longer processes if short processes keep arriving. This experiment implements SJF with arrival times, requiring the CPU to select the shortest available job at each decision point.

Procedure

1. Input the number of processes, Arrival Time (AT), and Burst Time (BT) for each.
2. Track completed processes. At each step, from all processes that have arrived ($AT \leq \text{current_time}$) and are not completed, select the one with the smallest BT.
3. If a process is found, execute it:
 - Update current_time by adding its BT.
 - Calculate TAT ($\text{current_time} - AT$), WT ($TAT - BT$), and CT.
 - Mark the process as complete.
 - Record the process ID and time in the Gantt chart.
4. If no process is available, increment current_time (CPU idle).
5. Repeat until all processes are completed. Calculate and print averages.

Program Code

```
#include <bits/stdc++.h>
#include <iomanip>
using namespace std;

int main() {
    int n;
```

```

cout << "Enter number of processes: ";
cin >> n;
int at[100], bt[100], tat[100], wt[100], completed[100] = {0};
int gantt[100], time_mark[101];
cout << "Enter Arrival Time and Burst Time\n";
for(int i=0; i<n; i++) {
    cout << "P" << i+1 << ": ";
    cin >> at[i] >> bt[i];
}

int current_time = 0, completed_count = 0, k = 0;
time_mark[0] = 0;
while(completed_count < n) {
    int idx = -1, min_bt = 9999;
    for(int i = 0; i < n; i++) {
        if(at[i] <= current_time && !completed[i] && bt[i] < min_bt) {
            min_bt = bt[i];
            idx = i;
        }
    }
    if(idx != -1) {
        current_time += bt[idx];
        tat[idx] = current_time - at[idx];
        wt[idx] = tat[idx] - bt[idx];
        completed[idx] = 1;
        gantt[k] = idx;
        time_mark[k+1] = current_time;
        k++; completed_count++;
    } else {
        current_time++;
    }
}

float avgWT = 0, avgTAT = 0;
cout << "\nProcess\tAT\tBT\tWT\tTAT\n";
for(int i=0; i<n; i++) {
    cout << "P" << i+1 << "\t" << at[i] << "\t" << bt[i] << "\t" << wt[i] << "\t" << tat[i] << endl;
    avgWT += wt[i]; avgTAT += tat[i];
}
cout << "\nAverage WT = " << avgWT/n << "\nAverage TAT = " << avgTAT/n;
cout << "\n\nGantt Chart:\n";

```

```

for(int i=0; i<k; i++) cout << "| P" << gantt[i]+1 << " ";
cout << "\n";
for(int i=0; i<=k; i++) cout << time_mark[i] << " ";
cout << endl;
return 0;
}

```

Input and Output:

```

Enter number of processes: 4
P1: 0 6
P2: 1 8
P3: 2 7
P4: 3 3

```

Output:

```

Process AT      BT      WT      TAT
P1      0       6       0       6
P2      1       8      15      23
P3      2       7       7      14
P4      3       3       3       6

Average WT = 6.25
Average TAT = 12.25

Gantt Chart:
| P1 | P4 | P3 | P2
0 6 9 16 24

```

Analysis / Discussion

For the given set of processes, SJF (Non-Preemptive) produced an Average WT of 6.25 and Average TAT of 12.25. In comparison, FCFS would yield an Average WT of 8.75 and TAT of 14.75. This proves SJF is optimal for reducing average waiting time. However, SJF required checking for available processes at each time unit, making it more complex than FCFS. A potential drawback is starvation; if a stream of short jobs keeps arriving, P2 could wait indefinitely.

Conclusion

The SJF (Non-Preemptive) algorithm was successfully implemented. The results confirm that SJF provides significantly lower average waiting and turnaround times compared to FCFS for the given workload. However, the algorithm's complexity increases with the need to sort and track available processes, and it can lead to starvation for longer processes.